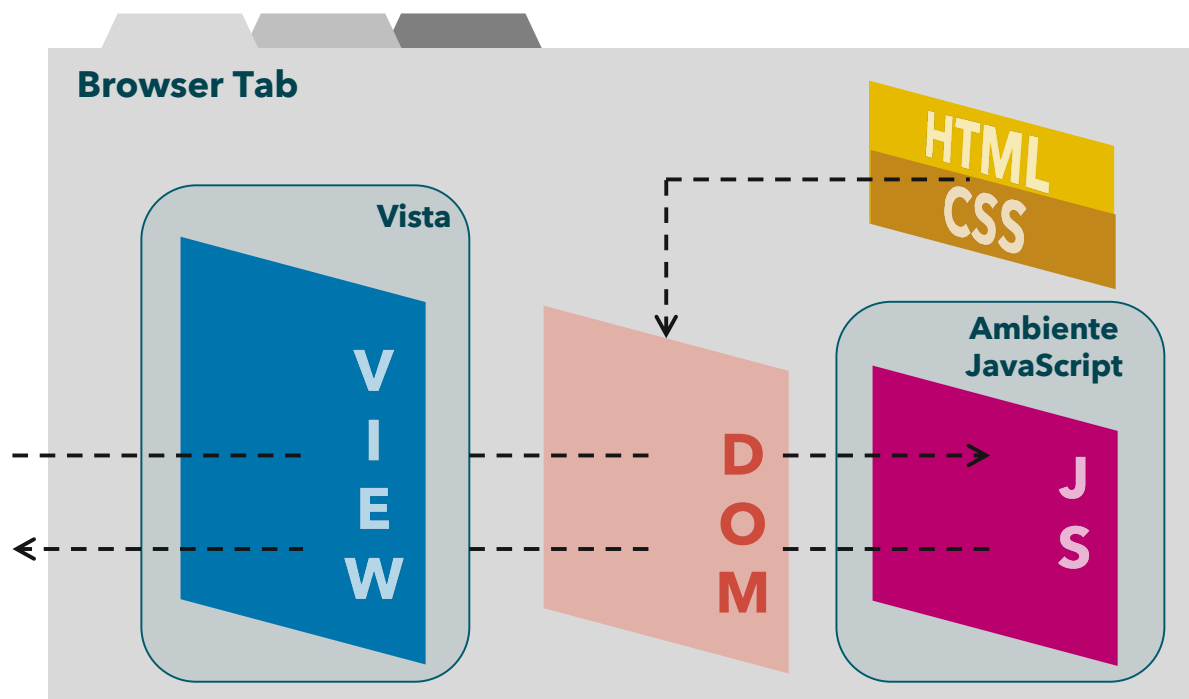




2 - La base dello sviluppo client-side: UIs con HTML e CSS

Tecnologie Web MFN0634 - © 2024 Claudia Picardi

Il browser come ambiente di esecuzione UI View & UI Logic



Il browser come ambiente di esecuzione – I

Abbiamo già visto che il client-side di una web app è costituito da:

- una o più pagine HTML
- ciascuna corredata da uno o più fogli di stile (CSS - Cascade Style Sheets)
- ciascuna associata a zero* o più moduli JavaScript

* una pagina senza moduli JavaScript è passiva (non interattiva) e di per sé non costituisce un'applicazione; tuttavia se un'applicazione è costituita da più pagine, è possibile che alcune di queste siano di sola lettura per l'utente.

È importante però sapere che il browser vede **ogni pagina** come a sé stante; ciascuna pagina viene infatti visualizzata/eseguita in una scatola chiusa (*sandbox*) che viene creata appositamente al caricamento della pagina da server.

Quindi, se la pagina viene ricaricata la sandbox viene distrutta e ne viene inizializzata una nuova: la memoria dell'applicazione in esecuzione viene perduta.

Si può ovviare parzialmente a questo problema utilizzando i meccanismi di storage locali del browser che possono essere condivisi fra più pagine **a patto che abbiano la stessa origine**, ossia provengano dallo stesso server. Si tratta però di un meccanismo con diversi limiti.

Se il client-side è costituito da più pagine, è compito del server-side gestire il flusso di informazioni fra una pagina e l'altra (o fra più caricamenti di una stessa pagina). Vedremo meglio questo aspetto quando approfondiremo la programmazione lato server.

Il browser come ambiente di esecuzione – II

Quando, come risultato di una richiesta HTTP, il browser ottiene una **pagina HTML + CSS + JS**, esso usa **HTML** e **CSS** per inizializzare la **vista** della pagina, ossia la finestra che l'utente vede e con cui può interagire.

Il documento HTML

Ha il ruolo di definire la **struttura logica** della pagina e la **semantica** delle sue diverse parti: in quante e quali sezioni è divisa, quali aree sono deputate a menu di navigazione e quali ad articoli di testo, il ruolo degli elementi di testo (titolo, paragrafo, elenco puntato o numerato...), quali sono gli eventuali contenuti multimediali che compongono la pagina, gli eventuali elementi di input (caselle di testo, pulsanti, menu a tendina...) che permettono all'utente di fornire informazioni... e così via. Inoltre l'HTML definisce i **collegamenti ipertestuali** che permettono di spostarsi ad altre pagine (o ad altre sezioni della stessa pagina).

I fogli di stile CSS

Sono insiemi di regole che definiscono l'aspetto grafico degli elementi che HTML struttura logicamente. Tipo e colore dei caratteri, colore di sfondo, dimensione e collocazione delle diverse sezioni: tramite i CSS è possibile controllare l'aspetto grafico della UI.

Il browser utilizza l'HTML e il CSS per inizializzare il **DOM (Document Object Model)** della pagina.

Il Document Object Model

È la struttura dati che mantiene in memoria tutte le informazioni sulla visualizzazione della pagina. Viene **inizializzata** a partire da HTML e CSS ma nel corso della sua vita può essere **modificata**. La vita del DOM termina quando la pagina viene chiusa o ricaricata.

HTML – Un linguaggio di annotazione - I

HTML nasce dall'esigenza di descrivere la struttura e l'aspetto di un documento multimediale ed ipertestuale contenente testo, immagini e opzionalmente altri *media* riproducibili (audio, video) utilizzando un formato di *testo semplice* (tecnicamente, la codifica è UTF-8), e disponendo delle URL dei file contenenti gli altri *media* che devono essere presenti sulla pagina.

Dunque HTML non è un **linguaggio di programmazione**, in quanto non istruisce algoritmicamente un agente di calcolo. HTML è piuttosto un **linguaggio di annotazione (markup)** in quanto parte da un testo e lo **suddivide** di **elementi** che lo strutturano logicamente e semanticamente.

Ogni **elemento** in HTML è fondamentalmente caratterizzato da:

- **Un tipo**
- **Un contenuto***, che può essere composto di testo e/o altri elementi
- **Una serie di attributi opzionali, che dipendono dal tipo di elemento**

La specifica formale dell'HTML definisce quali tipi di elementi esistono, qual è il loro «significato inteso», quale contenuto ci si aspetta che abbiano, e quali sono gli attributi aggiuntivi che è possibile specificare per ciascun tipo di elemento.

Il browser usa la **strutturazione** del documento (che è una struttura ad albero) per organizzarne la visualizzazione. Il «significato inteso» delle diverse tipologie di elementi, così come definito nella specifica formale dell'HTML, ha ovviamente un ruolo in questo, anche se come vedremo tramite i CSS è possibile alterare del tutto l'aspetto di un HTML.

*esistono alcuni tipi di elementi che non prevedono un contenuto.

HTML

HyperText Markup Language

Super Mario World [titolo]

Super Mario World, conosciuto come Super Mario World - Super Mario Bros. 4 in Giappone, è un videogioco a piattaforme per Super Nintendo Entertainment System (SNES) sviluppato da Nintendo Entertainment Analysis & Development e pubblicato da Nintendo in Giappone il 21 novembre 1990, in Nord America il 13 agosto 1991 e in Europa l'11 aprile 1992. È il sesto capitolo della serie Super Mario, parte del più ampio franchise di Mario.

[immagine: sm.jpeg]

Yoshi

Yoshi divenne uno dei personaggi più importanti della serie di Mario, riapparso nei successivi giochi di Super Mario e in quasi tutti gli spin-off e videogiochi sportivi di Mario.

Yoshi appare come il personaggio giocatore principale nel prequel di Super Mario World del 1995 #1#Super Mario World 2: Yoshi's Island#1#, che ha contribuito alla nascita di numerosi videogiochi incentrati sul personaggio.

Collegamenti:

l'area evidenziata da #1# deve collegare alla URL:

https://it.wikipedia.org/wiki/Super_Mario_World_2:_Yoshi%27s_Island

HTML

HyperText Markup Language

Super Mario World

Super Mario World, conosciuto come Super Mario World - Super Mario Bros. 4 in Giappone, è un videogioco a piattaforme per Super Nintendo Entertainment System (SNES) sviluppato da Nintendo Entertainment Analysis & Development e pubblicato da Nintendo in Giappone il 21 novembre 1990, in Nord America il 13 agosto 1991 e in Europa l'11 aprile 1992. È il sesto capitolo della serie Super Mario, parte del più ampio franchise di Mario.



Yoshi

Yoshi divenne uno dei personaggi più importanti della serie di Mario, riapparendo nei successivi giochi di Super Mario e in quasi tutti gli spin-off e videogiochi sportivi di Mario.

Yoshi appare come il personaggio giocante principale nel prequel di Super Mario World del 1995 [Super Mario World 2: Yoshi's Island](#), che ha contribuito alla nascita di numerosi videogiochi incentrati sul personaggio.

HTML

HyperText Markup Language

Yoshi

Yoshi divenne uno dei personaggi più importanti della serie di Mario, riapparendo nei successivi giochi di Super Mario e in quasi tutti gli spin-off e videogiochi sportivi di Mario.

Yoshi appare come il personaggio giocante principale nel prequel di Super Mario World del 1995 [Super Mario World 2: Yoshi's Island](#), che ha contribuito alla nascita di numerosi videogiochi incentrati sul personaggio.



Super Mario World

Super Mario World, conosciuto come Super Mario World - Super Mario Bros. 4 in Giappone, è un videogioco a piattaforme per Super Nintendo Entertainment System (SNES) sviluppato da Nintendo Entertainment Analysis & Development e pubblicato da Nintendo in Giappone il 21 novembre 1990, in Nord America il 13 agosto 1991 e in Europa l'11 aprile 1992. È il sesto capitolo della serie Super Mario, parte del più ampio franchise di Mario.

HTML

HyperText Markup Language

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Pagina di Esempio</title>
</head>
<body>
<section>
  <div>
    <h1>Super Mario World</h1>
    <p>Super Mario World, conosciuto come Super Mario World - Super Mario
      Bros. 4 in Giappone, è un videogioco a piattaforme per
      <em>Super Nintendo Entertainment System (SNES)</em>
      sviluppato da Nintendo Entertainment Analysis & Development...
    </p>
    
  </div>
  <div>
    <h2>Yoshi</h2>
    ...
  </div>
</section>
</body>
```

HTML

Sintassi di un elemento

Per definire un elemento di tipo XYZ si usa la seguente sintassi:

<XYZ> contenuto **</XYZ>**

tag di apertura: il nome identificativo del tipo di elemento, racchiuso fra parentesi angolari (simbolo del minore < e simbolo del maggiore >)

contenuto: una sequenza di testo e altri elementi HTML

tag di chiusura: analogo al tag di apertura, ma con uno slash / prima del nome del tipo di elemento

HTML

Attributi di un elemento

Un elemento può avere attributi con uno specifico valore.

gli attributi e relativi valori sono indicati «all'interno» del tag di apertura (dopo il nome identificativo, prima di chiudere la parentesi angolare); è possibile dichiararne uno, nessuno o molti, ma alcuni elementi hanno attributi che sono di fatto necessari.

`<XYZ attrib=val> contenuto </XYZ>`

nome dell'attributo: la specifica HTML dichiara quali attributi hanno senso (e qual è il senso inteso) per ciascun tipo di elemento. Attributi non presenti nella specifica vengono ignorati dal browser, eventuali funzioni Javascript associate all'elemento però possono sfruttare attributi custom definiti dal programmatore; questi attributi per convenzione hanno un nome della forma «data-*»

valore: il valore che si vuole assegnare all'attributo in questione; la specifica HTML dichiara quali tipi di valori sono validi per ciascun attributo. Potremo trovare stringhe, numeri interi o true/false

HTML

Struttura di un documento

La struttura di base di un documento HTML è fissa

```
<!DOCTYPE html>
<html lang="en">
  <head>
    ...
  </head>
  <body>
    ...
  </body>
</html>
```

preambolo: dichiara la versione della specifica HTML a cui si fa riferimento, esiste per retro compatibilità e di fatto è sempre uguale (alcuni documenti datati però potrebbero avere qui un'indicazione diversa)

intestazione: contiene **meta-informazioni** sul documento: servono al browser per visualizzato, ma non sono esse stesse contenuti visualizzati all'interno della pagina.

corpo: il contenuto effettivo del documento, strutturato in elementi come detto precedentemente; è ciò che viene effettivamente visualizzato all'interno della pagina

HTML – Tipi di elementi e loro contenuto - I

Alcuni tipi di elementi presenti nell'esempio di SuperMario sono HTML, HEAD, BODY, SECTION, ARTICLE, H1, P, EM.

Il tipo dell'elemento definisce il suo ruolo all'interno del documento. Ad esempio: l'elemento HTML indica **il documento in quanto tale** mentre gli elementi HEAD e BODY suddividono **ogni** documento HTML in due parti, l'**intestazione** (*head*) che contiene **metainformazioni**, ossia informazioni **sul** documento, e il **corpo** (*body*) che contiene il documento vero e proprio, ciò che viene effettivamente visualizzato nella finestra del browser.

Gli elementi di tipo SECTION indicano una suddivisione del documento in sezioni, mentre gli elementi DIV indicano una generica suddivisione del documento senza particolari connotazioni semantiche.

Altri elementi strutturali di questo tipo, non presenti in questo documento, sono NAV (che denota un'area dedicata alla navigazione del sito, quindi contenente link ad altre pagine) e ARTICLE (che denota un'area contenente un testo da leggere, usato per distinguere le sezioni testuali laddove la pagina contiene una UI complessa in cui molti elementi sono widget della UI e non materiale di lettura).

All'interno del testo, H1/H2 identificano titoli (*heading*) di primo e secondo livello (si arriva fino ad H6), P identifica il testo di un paragrafo, IMG indica un'immagine – di cui è necessario specificare la URL, deve cioè risiedere anch'essa come file indipendente su un web server.

EM racchiude un testo che deve essere enfattizzato (per default il browser lo mette in corsivo, ma questo comportamento può essere modificato con il foglio di stile). C'è anche l'analogo elemento STRONG che il browser per default visualizza in grassetto.

La specifica formale dell'HTML elenca tutti i tipi di elementi disponibili in HTML e il loro significato inteso (da cui solitamente consegue il modo in cui vengono visualizzati).

HTML – Tipi di elementi e loro contenuto - II

Come si vede nell'esempio, alcuni elementi (ad es. H1 o EM) hanno contenuto testuale, altri elementi invece hanno come contenuto dei sotto-elementi: l'elemento SECTION contiene due elementi DIV.

Ci sono elementi, come IMG, senza contenuto: notiamo tuttavia che l'elemento IMG deve **necessariamente** definire la URL dell'immagine a cui corrisponde come attributo.

Ci poi sono elementi, come P nell'esempio, che possono contenere un misto di testo e altri elementi. Più precisamente, P contiene nell'ordine: testo, elemento EM, testo.

È possibile inserire qualsiasi elemento dentro un altro? **Sì e no.**

Il browser farà del suo meglio per visualizzare la pagina, ma questo non significa che tutto ciò che possiamo fare abbia senso.

In prima approssimazione, possiamo suddividere gli elementi in due macro-categorie: elementi che definiscono un *riquadro* (detti elementi **a blocco**) e altri che invece seguono il *fluire del testo* (detti elementi **in linea**).

In un documento ben strutturato, gli elementi a blocco dovrebbero essere posizionati *solo dentro* ad altri elementi a blocco. Un elemento a blocco dentro un elemento in linea porterà a **comportamenti imprevisti** del browser. È opportuno, specie se usiamo HTML per progettare UI, che la visualizzazione sia ben strutturata, dunque prevedibile, controllabile, e indipendente dalle strategie compensative del browser.

HTML

Collegamenti ipertestuali

La struttura di base di un documento HTML è fissa

```
<a href="https://it.wikipedia.org/wiki/Super_Mario_World_2:_Yoshi%27s_Island">
  Super Mario World 2:Yoshi's Island
</a>
```

elemento: un collegamento ipertestuale si dichiara con l'elemento **A** (sta per **anchor**). L'elemento A può contenere sia **elementi in linea** che **elementi a blocco**.

hotspot: il contenuto dell'elemento A rappresenta il cosiddetto hotspot: ciò che permetterà all'utente, cliccandoci sopra, di spostarsi al documento destinazione

destinazione: la destinazione, o **target**, del collegamento è specificata tramite l'attributo **href**. Il valore di questo attributo è una stringa che deve corrispondere a una URL valida.

URL esterna: specifica il dominio e il path del documento destinazione (si presuppone che il dominio sia diverso da quello del documento in cui ci troviamo)

URL assoluta: rimanda a un documento nello stesso dominio; specifica il path completo del documento destinazione a partire dalla radice del dominio stesso, iniziando con lo slash (/)

URL relativa: rimanda a un documento nello stesso dominio; specifica il path relativo rispetto al documento attuale.

Elementi HTML – il testo

Ricapitoliamo qui molto brevemente i **principali e più usati** tag HTML. Come documentazione in rete fra gli altri è possibile utilizzare:

Mozilla developer network: <https://developer.mozilla.org/en-US/docs/Learn/HTML>

W3Schools: <https://www.w3schools.com/html/>

Elementi a blocco relativi al testo:

- **P** – paragrafo
- **H1, H2, H3, H4, H5, H6** – titoli di vari livelli ad importanza decrescente
- **UL** – lista a punti; contiene una sequenza di elementi **LI** ciascuno dei quali corrisponde a un elemento della lista. L'attributo **type** (valore «circle», «disc» o «square») permette di modificare il tipo di punto elenco di default.
- **OL** – lista numerata; contiene una sequenza di elementi **LI** ciascuno dei quali corrisponde a un elemento della lista. L'attributo **type** (valori «a», «A», «i», «I», «l») permette di scegliere il tipo di numerazione fra alfabetica minuscola/maiuscola, romana minuscola/maiuscola, numeri interi. L'attributo **start** permette di far specificare il valore da cui fa partire la numerazione.

Elementi in linea relativi al testo:

- **EM** – testo da enfatizzare; per default in corsivo
- **STRONG** – testo da enfatizzare **molto**; per default in grassetto
- **SPAN** – testo che si desidera in qualche modo distinguere; l'aspetto grafico verrà successivamente stabilito con i CSS. Il browser di default non lo distingue dal resto.

Elementi HTML – tabelle

In alcuni documenti può essere utile organizzare i contenuti in forma tabellare, in particolare se si vogliono presentare dati in forma organizzata, con righe/colonne. Insomma: tutti quei casi in cui useremmo una tabella in un documento, in un articolo, in una relazione tecnica...

Attenzione: non utilizzare le tabelle per strutturare la **disposizione** dei contenuti sulla pagina! Per questo come vedremo in seguito ci sono gli **elementi strutturali** che, in combinazione con i fogli di stile CSS, permettono di ottenere qualsiasi disposizione si voglia.

- **TABLE** – l'elemento che definisce l'intera tabella. Al suo interno contiene una sequenza di righe. Ciascuna riga è identificata tramite l'elemento **TR**.
- **TR** – rappresenta una riga di una tabella. Al suo interno contiene una sequenza di celle, ciascuna identificata da un elemento **TD**.
- **TD** – rappresenta una cella di una riga di tabella. Gli attributi **rowspan** e **colspan** (numeri interi > 0) permettono di stabilire se la cella spazia su più righe o su più colonne; se non specificati valgono entrambi 1. Idealmente la somma dei **colspan** delle celle di una riga dovrebbe essere costante per tutte le righe della tabella.
- **TH** – rappresenta una cella della tabella che ha il ruolo di intestazione di riga o di colonna; l'attributo **scope** (valore «row» oppure «col») permette di distinguere i due casi.
- **THEAD, TBODY, TFOOT**: è possibile usarli per raggruppare le righe di una tabella in tre parti diverse, identificate rispettivamente come l'intestazione, il corpo principale e il piè di pagina della tabella.

Elementi HTML – contenuti multimediali

- **IMG** – un'immagine. Si tratta di un elemento **in linea**. L'attributo **src** specifica l'URL dell'immagine da mostrare; può essere una URL esterna, assoluta o relativa; gli attributi **width** e **height** permettono di specificare la larghezza e l'altezza della visualizzazione dell'immagine ed è consigliabile specificarli in modo che il browser possa impostare la pagina senza dover aspettare di aver caricato l'immagine.

Attenzione: l'immagine viene comunque scaricata dal server al browser nella sua dimensione originale, e solo successivamente ridimensionata dal browser per corrispondere ai parametri specificati. Quindi se l'obiettivo nel «rimpicciolire» un'immagine è di limitare il traffico di rete, l'uso degli attributi di IMG non assolve allo scopo. È necessario in quel caso tenere sul server più versioni dell'immagine a dimensioni diverse, e utilizzare la URL di quella che si vuole effettivamente utilizzare.

- **AUDIO** – permette di mostrare all'interno della pagina («embedded», ossia incastonato fra gli altri elementi) un piccolo player che riproduce l'audio indicato. Anche in questo caso l'attributo **src** indica la URL del file audio. Vari altri attributi permettono di configurare varie opzioni; fra gli altri, **autoplay** (true/false) indica se far partire automaticamente l'audio al termine del download, **controls** (true/false) indica se mostrare all'utente i comandi del player.
- **VIDEO** – analogo ad AUDIO, ma per i video. In questo caso sono disponibili anche gli attributi **width** e **height** per regolare le dimensioni del riquadro di riproduzione. Su questo valgono le stesse considerazioni fatte per le immagini.

Elementi HTML – elementi strutturali

L'uso che si fa degli elementi strutturali può variare molto a seconda dello scopo della pagina HTML che si sta realizzando. In particolare possiamo distinguere un uso «redazionale» (un documento, un articolo scientifico, un quotidiano o un magazine, un blog – l'interazione con l'utente è minima, ci si aspetta che l'utente legga o visualizzi i contenuti e navighi da una pagina all'altra o da una sezione all'altra della stessa pagina) e un uso «applicativo» (la pagina rappresenta l'interfaccia grafica di un'applicazione, di un e-commerce, di una piattaforma social, ecc – l'interazione con l'utente riveste un ruolo importante e l'utente ha un ruolo attivo nello scegliere opzioni, caricare contenuti, fornire informazioni, ecc.).

- **DIV** – denota una suddivisione strutturale del documento senza particolari significati associati a priori; riveste un ruolo importante quando si desidera dare un preciso **layout** (= disposizione degli elementi) alla pagina, diverso da quello di default. Pressoché indispensabile nella progettazione di UI.

Diremo di più sui possibili layout di una pagina più avanti, dopo aver parlato di CSS.

- **SECTION** – usato per suddividere una pagina in più sezioni indipendenti
- **ARTICLE** – usato per contraddistinguere all'interno della pagina un articolo che potrebbe anche essere presentato indipendentemente dal resto della pagina
- **NAV** – usato per distinguere in una pagina un'area dedicata a un menu di navigazione
- **HEADER** – distingue l'intestazione visibile di una pagina o di una sezione, di solito contiene cose come il titolo principale, il logo (ma spesso al suo posto si usa un qualunque «DIV»). Da non confondersi con **HEAD** che contiene invece la meta-intestazione del documento, ossia informazioni usate dal browser ma non visualizzate all'utente.
- **FOOTER** – contraltare di **HEADER**, distingue il piè di pagina, tipicamente contiene i contatti dell'azienda o dell'autore del sito, informazioni relative ai Cookies, etc.

Elementi HTML – moduli (form)

L'HTML puro prevede la possibilità di inserire nella pagina dei **moduli** che permettano all'utente di inviare dati al web server (naturalmente, questo ha senso se il web server li riceve con una server-side app in grado di elaborarli e/o memorizzarli; altrimenti i dati inviati vengono perduti).

Attenzione: parliamo qui esclusivamente di **form HTML gestiti dal browser**.

Vedremo in seguito che è possibile costruire **form HTML gestiti da codice**

JavaScript, con una maggiore flessibilità di comportamento.

Un modulo in HTML è rappresentato da un **elemento di tipo FORM** che contiene al suo interno uno o più **elementi di input** (pulsanti, check box, caselle di testo, ecc.).

Il tag <FORM>...</FORM>

Il tag FORM definisce, attraverso i suoi attributi, il comportamento del modulo.

- attributo **action**: la URL (esterna, relativa o assoluta) a cui i dati raccolti devono essere inviati
- attributo **method**: il method HTTP (GET o POST) da usare per inviare i dati inseriti dall'utente. Se si usa il metodo GET, i dati vengono codificati come **querystring** nella URL; se si usa il metodo POST, i dati vengono inseriti nel **body** della request con Content-Type **application/x-www-form-urlencoded**

Indipendentemente dal fatto che i dati vengano codificati come querystring di una GET o come body di una POST, sono rappresentati nella forma

nome₁=valore₁&nome₂=valore₂&...&nome_K=valore_K

I nomi dei campi sono desunti dagli elementi di input contenuti nel form, i valori sono quelli inseriti dall'utente.

Elementi HTML – moduli (form) - II

Gli elementi di input

Non ci sono vincoli su quali elementi HTML possano stare all'interno di un form, che può dunque anche contenere un HTML piuttosto articolato. Se dobbiamo costruire un modulo perché l'utente inserisca dei dati avremo bisogno tuttavia di prevedere degli elementi di input.

La maggior parte degli elementi di input può essere inserita tramite il tag INPUT; esso prevede un attributo **type** che decide di **quale** tipo di input si tratti.

Importante: tutti gli elementi INPUT **richiedono obbligatoriamente** l'attributo **name**: il valore di tale attributo corrisponde al **nome** del campo corrispondente che verrà codificato nella richiesta HTTP.

Vediamo gli INPUT più comuni;

- **<INPUT type="text"/>**: una **casella di testo**. Alcuni attributi aggiuntivi: **size** (la dimensione della casella di testo in numero di caratteri), **value** (se vogliamo che ci sia già un valore presente nella casella, che l'utente possa mantenere o modificare), **placeholder** (se vogliamo che nella casella sia presente un'indicazione su cosa inserire, che però non conta come valore, ma è solo un segnaposto e scompare non appena l'utente inizia a digitare)
- **<INPUT type="checkbox"/>**: una **checkbox**. L'attributo **value** esprime il valore da riportare nella richiesta HTTP in caso la checkbox sia contrassegnata. L'attributo **checked**, se presente, indica che la checkbox appare inizialmente già contrassegnata.
- **<INPUT type="radio"/>**: un **radio-button**. Solitamente i radio-button si presentano a gruppi e solo uno dei radio-button di uno stesso gruppo può essere selezionato. In questo caso l'attributo **name** deve avere lo stesso valore per tutti i radio-button di uno stesso gruppo; l'attributo **value** esprime il valore che verrà riportato in base a quale radio-button è effettivamente selezionato. Anche qui l'attributo **checked**, se presente, individua quale dei radio-button di un gruppo è inizialmente selezionato. **[...continua]**

Elementi HTML – moduli (form) - II

[...continua] Gli elementi di input

Oltre a quelli descritti qui ci sono moltissimi altri tipi di INPUT; per un elenco completo si può consultare:

<https://developer.mozilla.org/en-US/docs/Web/HTML/Element/input>

Ci sono tuttavia altri due tipi di elementi HTML molto comunemente usati all'interno di un FORM:

- **<LABEL for="inputid">...</LABEL>**: una **etichetta esplicativa** associata ad un determinato elemento INPUT (per checkbox e radio-button è essenziale che ci sia, mentre per le textbox può essere omessa se c'è un placeholder abbastanza esplicativo). L'uso di LABEL prevede che il corrispondente INPUT abbia un valore assegnato per l'attributo **id**. Quel medesimo valore dovrà essere fornito nell'attributo **for** della LABEL, per abbinarla all'INPUT corretto.
- **<SELECT name="aaa">...</SELECT>**: denota un menu a tendina, all'interno del quale l'utente potrà selezionare una fra diverse opzioni. L'elenco delle opzioni è riportato **all'interno** dell'elemento SELECT; ciascuna opzione è un elemento di tipo OPTION:
<OPTION value="opt1">Prima opzione</OPTION>
<OPTION value="opt2">Seconda opzione</OPTION>

L'attributo **name** è associato all'elemento SELECT; l'attributo **value** in questo caso va specificato sulle singole OPTION; in base alla scelta dell'utente verrà riportato il **value** corrispondente nella richiesta HTTP.

Elementi HTML – moduli (form) - III

Submit e reset

Un form in HTML puro (a differenza di uno gestito tramite JavaScript) deve necessariamente prevedere un pulsante che, se premuto, provoca l'invio dei dati al web server, secondo quanto specificato negli attributi **action** e **method** del tag FORM.

Questo si ottiene inserendo all'interno del form un pulsante di **submit**:

`<INPUT type="submit" value="Premi qui"/>` in questo caso il campo di input non fornisce un dato da inviare al web server, ma provoca l'effettiva azione dell'invio. L'attributo **value** definisce il testo che appare sul pulsante.

Parallelamente, è possibile (ma non necessario) inserire nel form un pulsante di **reset** che ha l'effetto di riportare tutti gli input allo stato iniziale:

`<INPUT type="reset" value="Ricomincia"/>`

CSS Cascading Style Sheets

```
html {
  background-color: #b7dab7;
}

body {
  font-size: 14pt;
  max-width: 800pt;
  margin-left: auto;
  margin-right: auto;
}

h1 {
  background-color: darkslategrey;
  color: bisque;
  padding: 10px;
  font-weight: normal;
  font-size: 2rem;
  font-family: sans-serif;
}

...
```

```
img {
  border: 2px solid;
  border-color: #ff006a;
  max-width: 300px;
  float: left;
  margin-right: 10px;
  margin-bottom: 10px;
}

h2 {
  color: #ff006a;
  background-color: rgba(255,0,106,0.1);
}

a {
  text-decoration: none;
  color: teal;
  background-color: #dfff76;
}
```

CSS

Le regole di stile

Lo stile **grafico** con cui un documento HTML è reso a video è descritto tramite **regole** raccolte in uno o più file con estensione **.css**

selettore: filtra gli elementi a cui la regola deve essere applicata. Il tipo più semplice di selettore è il nome di un elemento HTML: la regola verrà applicata a tutti gli elementi di quel tipo.

```
h1 {  
  background-color: darkslategrey;  
  color: bisque;  
  padding: 10px;  
  font-family: sans-serif;  
}
```

blocco delle dichiarazioni:
elenca le dichiarazioni di stile che dovranno essere applicate agli elementi selezionati dal selettore, fra parentesi graffe.

dichiarazione:

esprime un singolo elemento di stile ed è formulata come: **proprietà: valore;**
La specifica CSS definisce un lunghissimo elenco di proprietà che permettono di controllare interamente l'aspetto del documento.
Definisce anche, per ogni proprietà, quali tipi di valori essa ammette.

CSS

Selettori

Il linguaggio CSS è reso estremamente **flessibile** dalla possibilità di specificare dei selettori che sono delle vere e proprie query sul documento html

– **selettori per elemento**

– **selettori per classe**

la classe di un elemento può essere specificata tramite l'attributo html **class**

– **selettori in base alla posizione nel documento**

è possibile selezionare gli elementi in base al fatto che si trovino all'interno di un altro elemento, prima di esso, dopo di esso, o in base alla posizione che occupa all'interno dell'elemento genitore

– **selettori in base allo stato di un elemento**

ad esempio, se l'elemento è in stato di **hover** (l'utente ci sta passando sopra con il mouse) o, nel caso di un link ipertestuale, se è stato **visitato**, o nel caso di un bottone se è **attivo**...

CSS

Assegnare regole di stile a un HTML

Foglio di stile **esterno**: un documento separato, collegato specificato nel blocco **head** dell'html tramite l'elemento **link**

```
<head> ...  
  <link rel="stylesheet" href="styles/style.css">  
</head>
```

Foglio di stile **interno**: le regole vengono scritte direttamente nel blocco **head** all'interno di un apposito elemento **style**

```
<head> ...  
  <style> [regole css] </style>  
</head>
```

Foglio di stile **in linea**: nell'HTML un insieme di **dichiarazioni di stile** vengono associate a un singolo elemento tramite l'**attributo style**

```
...<h1 style="color: red; font-size: 15pt;">  
  Bla bla  
</h1>
```

CSS

Regole a cascata

Cosa succede se a uno stesso elemento si applicano due regole con impostazioni di stile **contrastanti**? (ossia: stesso attributo, valori diversi)

Uno stesso html può collegare più fogli di stile esterni, può avere sia stili esterni che interni che in linea...

Cascata	Specificità	Ereditarietà
Conta l'ordine in cui le regole vengono ricevute dal browser: se hanno la stessa specificità vince la regola definita per ultima. Quindi conta l'ordine in cui i fogli di stile esterni sono specificati, i fogli di stile interni vengono considerati dopo quelli esterni, e le dichiarazioni in linea vengono considerate per ultime.	La specificità di una regola rispetto a un elemento dipende da quanto precisamente il selettore individua l'elemento in questione. La specificità viene calcolata con un algoritmo di cui non vedremo i dettagli. Le dichiarazioni di stile più specifiche in assoluto sono quelle in linea !	Alcune proprietà di stile vengono ereditate di default dal proprio genitore, o i loro valori possono essere calcolati sulla base del valore specificato per il genitore. Anche se questo non influenza l'algoritmo che sceglie quali impostazioni di stile applicare, ha un effetto sul risultato e va tenuto in conto quando i nostri CSS non sembrano comportarsi come vorremmo...

Proprietà e valori compositi

Alcune proprietà CSS sono in realtà delle scorciatoie per andare ad impostare in un'unica regola più proprietà diverse.

Vediamo un esempio:

- i margini del box che circonda un elemento possono essere impostati con quattro proprietà:

```
margin-left: 20px; margin-right: 10px; margin-top: 5px; margin-bottom: 0;
```

- la singola proprietà **margin** permette di impostare tutti e 4 i valori con una sola regola:

```
margin: 5px 10px 0 20px; /* top *** right *** bottom *** left */
```

- è però possibile usare **margin** assegnando un unico valore, che sarà lo stesso per tutti e 4 i margini:

```
margin: 10px; /* top = right = bottom = left = 10px */
```

- Ed è anche possibile usare **margin** assegnando **due** valori, margini verticali e margini orizzontali:

```
margin: 10px 20px; /* top = bottom = 10px *** left = right = 20px */
```

CSS

Tipi di selettori

L'aspetto più sofisticato (e più potente) dei **CSS** è la possibilità di specificare **selettori compositi** e **complessi**

il **selettore** agisce come una vera e propria query, permettendo di raffinare moltissimo l'insieme di elementi a cui la regola verrà applicata

selettore: filtra gli elementi a cui la regola deve essere applicata.

```
h1 { ... }
```

CSS

Selettori semplici

per TAG	per CLASSE	per ID
<pre>img { /* stili */ } a { /* stili */ }</pre>	<pre>.articolo { /* stili */ } <div class="articolo"> ... </div></pre>	<pre>#utente { /* stili */ } <div id="utente"> ... </div></pre>
per ATTRIBUTO	per PSEUDOCLASSE	UNIVERSALE
<pre>[type="text"] { /* stili */ } [href*="example"] { /* stili */ }</pre>	<pre>:hover { /* stili */ } :first-child { /* stili */ }</pre>	<pre>* { /* stili */ }</pre>

CSS

Selettori composti

I selettori **compositi** si ottengono combinando per **giustapposizione** più selettori semplici

Verranno selezionati gli elementi che corrispondono a **tutti** i selettori giustapposti (la giustapposizione corrisponde ad un operatore **and**).

<pre>div.importante { /* stili */ }</pre>	Tutti gli elementi di tipo DIV che appartengono anche alla classe .importante
<pre>label[name="anno"] { /* stili */ }</pre>	Tutti gli elementi di tipo LABEL che hanno l'attributo name pari ad "anno"
<pre>.elenco.attivo { /* stili */ }</pre>	Tutti gli elementi (di qualsiasi tipo) che appartengono alla classe elenco e anche alla classe attivo . Il selettore universale è implicito
<pre>input[type="button"]:hover { /* stili */ }</pre>	Tutti i pulsanti (elementi di tipo INPUT con attributo type pari a "button»), ma solo quando ci passa sopra il mouse (:hover).

CSS

Selettori complessi

I selettori **complessi** sono sequenze di selettori semplici o composti, separati da opportuni **combinatori**

DISCENDENTE (spazio)

```
.articolo h1  
{ /* stili */ }
```

Tutti gli elementi **H1** che si trovano all'interno di un elemento con classe **articolo**

FIGLIO (>)

```
ul.menu > li  
{ /* stili */ }
```

Tutti gli elementi **LI** che si trovano **direttamente** all'interno di un elemento **UL** con classe **menu**

SUCCESSORE (+)

```
h1 + p  
{ /* stili */ }
```

L'elemento **P** che, all'interno del medesimo genitore, viene **immediatamente dopo** un elemento **H1**

FRATELLO (~)

```
.introduzione ~ div  
{ /* stili */ }
```

Tutti gli elementi **DIV** che, all'interno del medesimo genitore, vengono **dopo** un elemento con classe **introduzione**

CSS

Lista di selettori

È possibile applicare una regola a una **lista** di selettori, usando la **virgola** come separatore

Verranno selezionati gli elementi che corrispondono ad **almeno un** selettore nella lista (la lista corrisponde ad un operatore **or**).

```
h2, .capitolo > h1, input[type="text"]:hover  
{ /* stili */ }
```

Le regole si applicano:

- a tutti gli elementi di tipo **H2**
- a tutti gli elementi di tipo **H1** che si trovano immediatamente dentro a un elemento della classe **capitolo**
- a tutti gli elementi di tipo **INPUT** con attributo **type** pari a "text" ma solo quando il mouse vi passa sopra

I selettori CSS - I

Per scrivere un buon CSS è necessario usare sapientemente i selettori, in combinazione con l'HTML delle pagine per cui il CSS viene progettato. Possiamo suddividere i selettori in selettori semplici, selettori composti e selettori complessi. I selettori composti si ottengono giustapponendo in sequenza più selettori semplici. I selettori complessi si ottengono combinando selettori semplici o composti con opportuni operatori.

Ogni selettore, semplice, composto o complesso che sia, ha l'effetto di **selezionare** (come dice il nome) un ben preciso insieme di elementi dell'HTML, ai quali verrà applicata la regola a cui il selettore si riferisce.

I selettori CSS - II

Selettori semplici:

Selettori per TAG: il selettore coincide con un tag HTML e seleziona tutti gli elementi definiti con quel tag.

Selettori per CLASSE: decisamente la categoria più utilizzata. È possibile assegnare una **classe** a un elemento HTML tramite l'attributo **class**. Le «classi» dell'HTML non sono altro che stringhe liberamente stabilite da chi scrive il documento, che «etichettano» gli elementi con il preciso scopo di raggrupparli e renderli selezionabili nei CSS e, come vedremo in seguito, nel codice JavaScript. Una classe potrà servire ad esempio a identificare quale **SECTION** dell'HTML contiene il testo principale, o a distinguere quali elementi **LI** all'interno di una lista devono apparire evidenziati. Data una classe **pincopallino**, il selettore **«.pincopallino»** seleziona tutti gli elementi appartenenti a quella classe. Quindi il nome della classe è una stringa composta di caratteri alfanumerici e trattini (-), il selettore corrispondente si ottiene antepoendo un punto (.) al nome della classe.

Selettori per ID: è anche possibile assegnare un **id** univoco a un elemento HTML, tramite l'attributo **id**. Sarà poi possibile scrivere un selettore CSS che fa riferimento a quell'unico id premettendo all'id il simbolo hashtag (#). Quindi se un elemento ha id **pippo**, il selettore corrispondente è **#pippo**. Gli **id** sono meno utilizzati delle classi, principalmente per la difficoltà di garantirne l'univocità.

[...continua]

I selettori CSS - III

Selettori semplici:

[...continuazione]

Selettori per PSEUDOCLASSE: le pseudoclassi sono raggruppamenti **predefiniti** di elementi che vengono determinati dal browser in base al loro stato corrente (statico o dinamico). Avremo dunque pseudoclassi dinamiche come **hover** (il mouse sta passando sopra l'elemento), o statiche come **first-child** (l'elemento è il primo figlio dell'elemento contenitore). Il selettore si ottiene anteposto al nome della pseudoclasse i due punti (:). Per un elenco completo di pseudoclassi:

<https://developer.mozilla.org/en-US/docs/Web/CSS/Pseudo-classes>

Selettori per ATTRIBUTO: seleziona un gruppo di elementi HTML sulla base dei loro attributi e dei relativi valori. Il selettore per attributo è sempre racchiuso fra parentesi quadre e la sua forma è una delle due seguenti:

[**attr**] per selezionare tutti gli elementi che **dichiarano** l'attributo **attr**, indipendentemente dal valore

[**attr****OP**"**val**"] per selezionare tutti gli elementi in cui il valore dell'attributo **attr** **corrisponde** al valore **val**, dove il significato di «corrispondere» dipende dall'operatore **OP** utilizzato. Ad esempio [**attr****=**"**val**"] seleziona tutti gli elementi in cui **attr** **vale esattamente** **val**, mentre [**attr*******"**text**"] seleziona tutti gli elementi in cui il valore di **attrib** **contiene** la stringa **text**

Selettore UNIVERSALE: seleziona **tutti** gli elementi ed è (non sorprendentemente) identificato con un asterisco (*)

I selettori CSS - IV

Selettori composti

I selettori composti si ottengono per **giustapposizione** di uno o più selettori semplici: questo significa che i selettori semplici oggetto della composizione vengono **elencati** uno dopo l'altro senza alcuno spazio in mezzo.

L'effetto di questa composizione è quello di un operatore **and**: vengono selezionati per l'applicazione della regola quegli elementi che rispettano **tutti** i selettori semplici oggetto della composizione.

La composizione deve rispettare alcuni (pochi) vincoli:

- La composizione deve **necessariamente** prevedere un unico selettore per **tag**, oppure il selettore **universale**
- Il selettore per **tag**, se presente, deve essere il primo elemento della composizione
- Se invece si intende usare il selettore **universale**, anch'esso deve comparire come primo elemento della composizione, ma può anche essere **omesso**.

L'ordine in cui vengono giustapposti i selettori semplici non cambia il significato della composizione; tuttavia potrebbe influire sull'applicazione della **cascata**, in particolare per quanto riguarda la valutazione della **specificità** del selettore stesso.

I selettori CSS - V

Selettori complessi

I selettori complessi permettono di selezionare gli elementi html sulla base della loro posizione nella **struttura** del documento. Essi sono sequenze di due o più selettori semplici o composti separati da **combinatori**.

Formalmente, un selettore complesso ha la forma **S comb A** dove

- **S** può essere un selettore qualunque (semplice, composto o complesso)
- **comb** è un combinatore
- **A** è un selettore semplice o composto

Il vincolo di selezione espresso è «seleziona tutti gli elementi che rispettano il selettore **A** e che sono nella relazione espressa da **comb** con un elemento che rispetta il selettore **S**».

La cosa sarà più chiara esaminando i possibili combinatori:

spazio vuoto esprime la relazione «discendente di».

Il selettore complesso **S A** seleziona tutti gli elementi **A** e che sono discendenti di un elemento **S**.

> (maggiore) esprime la relazione «figlio di».

Il selettore complesso **S > A** seleziona tutti gli elementi **A** che sono figli di un elemento **S**.

~ (tilde) esprime la relazione «fratello successivo di».

Il selettore complesso **S ~ A** seleziona tutti gli elementi **A** che seguono (anche non consecutivamente) a un elemento **S**, condividendo con esso il medesimo genitore.

+ (tilde) esprime la relazione «prossimo fratello di».

Il selettore complesso **S + A** seleziona gli elementi **A** che seguono immediatamente un elemento **S**, condividendo con esso il medesimo genitore.

CSS Layout

Un aspetto tipografico che è essenziale poter gestire tramite CSS è il **layout**: la disposizione degli elementi nella pagina

TITOLO ARTICOLO 1

IMG 1

Testo testo testo testo testo
testo testo testo testo testo
testo testo testo testo testo

info articolo e autore

TITOLO ARTICOLO 2

IMG 2

Testo testo testo testo testo

CSS

Layout

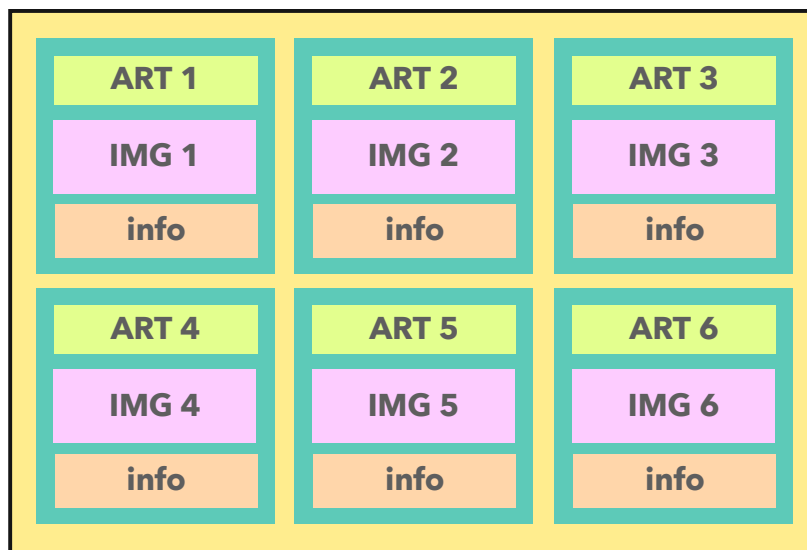
Un aspetto tipografico che è essenziale poter gestire tramite CSS è il **layout**: la disposizione degli elementi nella pagina



CSS

Layout

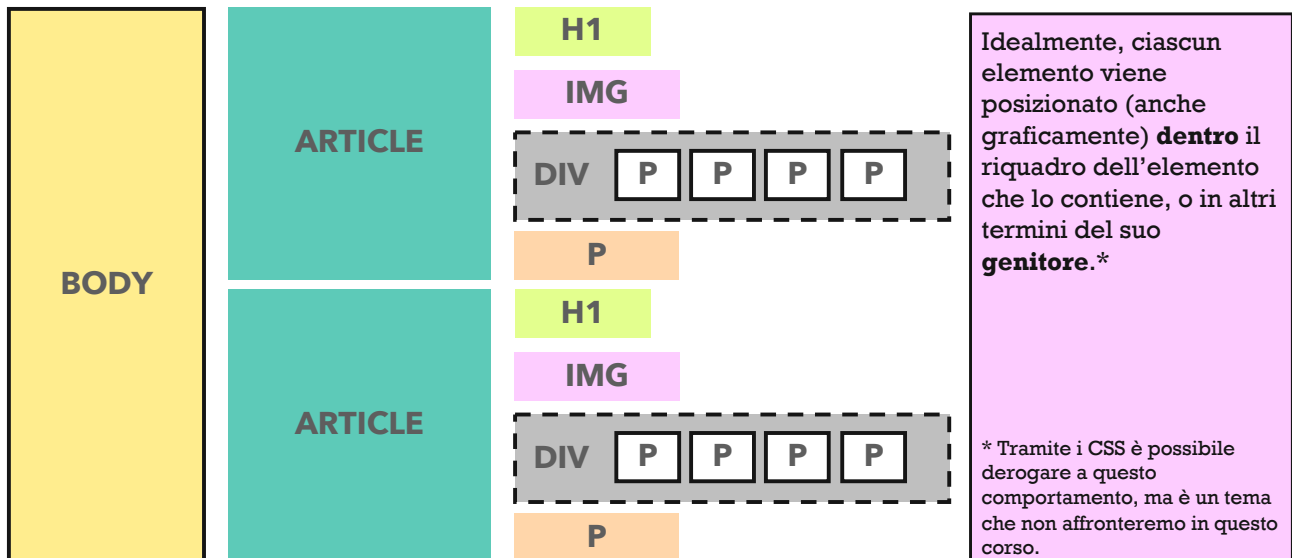
Un aspetto tipografico che è essenziale poter gestire tramite CSS è il **layout**: la disposizione degli elementi nella pagina



CSS

Layout e struttura dell'HTML

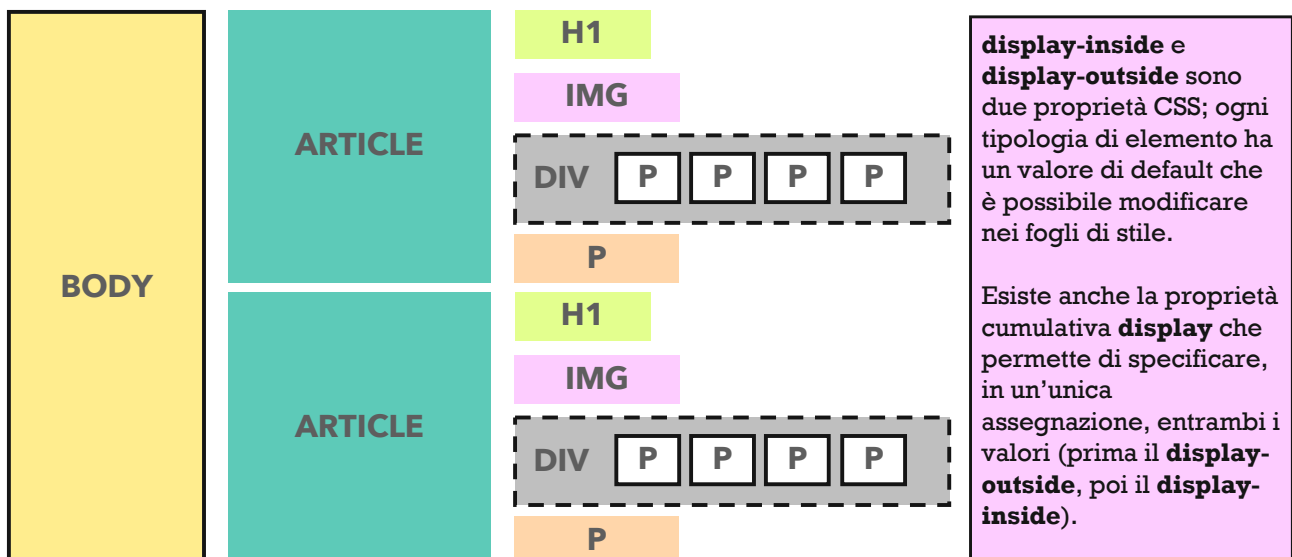
La disposizione degli elementi sulla pagina viene determinata dal browser e dipende innanzitutto dalla **struttura** dell'HTML, e in particolare dal fatto che gli elementi siano **contenuti** uno nell'altro, in un modello a **scatole cinesi**.



CSS

Layout e display

In secondo luogo, il browser considera il **display** di ciascun elemento (espresso tramite CSS): **display-inside**, ossia come l'elemento disporrà i propri contenuti; **display-outside**, ossia come l'elemento preferirebbe essere disposto nel proprio contenitore,



CSS: il layout degli elementi

È il browser a stabilire come disporre gli elementi sulla pagina sulla base di due fattori fondamentali:

1. La **struttura** della pagina, che, come abbiamo visto, può essere vista come una serie di scatole cinesi in cui ogni elemento è **contenuto** in un altro (tranne l'elemento radice, ossia **html**). Incidentalmente, la struttura a scatole cinesi è anche una struttura **ad albero**: il «contenitore» è il **nodo genitore** e ciò che è in esso contenuto sono i **nodi figli**. Vedremo che la struttura ad albero dell'HTML sarà fondamentale nell'ottenere una rappresentazione del documento consultabile e manipolabile tramite codice.
2. Lo stile di **display** associato a ciascun elemento. I contenuti di ciascun elemento (le «scatole al suo interno», per così dire) sono disposti secondo il suo **display-inside**, alcuni tipi di **display-inside** tengono conto dello stile **display-outside** dei propri figli (le «scatole interne»), per altri invece questa informazione non è rilevante. Ciascuno stile di **display-inside** può poi essere ulteriormente configurato attraverso altre proprietà CSS sia del genitore («scatola contenitore») che dei figli («scatole interne»).

La proprietà CSS **display** è una scorciatoia per esprimere entrambi i valori:

```
P { display: inline flow; } /* inline è il display-outside, flow è il display-inside */
```

Siccome ogni elemento ha un display-outside e un display-inside di default, è possibile utilizzare la proprietà display per esprimere solo uno dei due valori.

```
P { display: block; } /* block è il display-outside, il display-inside è flow di default */
```

```
P { display: flex; } /* flex è il display-inside, il display-outside di default di un P è block */
```

CSS display-inside: flow

Il display-inside di tipo **flow** è quello di **default** e dispone gli elementi come si farebbe per un articolo di rivista o un documento di testo.

TITOLO ARTICOLO 1

IMG 1

Testo testo testo testo testo
testo testo testo testo testo
testo testo testo testo testo

info articolo e autore

TITOLO ARTICOLO 2

IMG 2

Testo testo testo testo testo

CSS

display-inside: **flow**

Il display-inside di tipo **flow** è quello di **default** e dispone gli elementi come si farebbe per un articolo di rivista o un documento di testo.



In questo caso è rilevante il **display-outside** dei contenuti.

Un elemento interno con **display-outside** pari a **block** è rappresentato in un riquadro che occupa tutto il contenitore in orizzontale, mentre in altezza si estende quanto gli serve per rappresentare tutti i propri contenuti. Si pensi per analogia a un paragrafo in un testo.

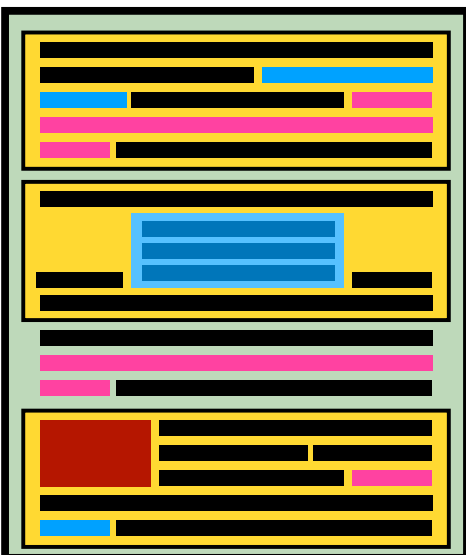
Un elemento interno con **display-outside** pari a **inline** è rappresentato come una sequenza di testo o simboli che scorre (almeno per le lingue indoeuropee) da sinistra a destra, su più righe orizzontali, che occupano la larghezza del contenitore; nel caso di due elementi **inline** consecutivi il secondo inizierà nel punto in cui finisce il primo.

È possibile portare un elemento interno «fuori» dal flusso delle righe, costringendole a girarci intorno: questo si ottiene impostando per la proprietà **float** un valore pari a **left** o **right**.

CSS

display-inside: **flow-root**

Il display-inside di tipo **flow-root** è una variante di **flow** che diventa interessante se applicata a un elemento con display-outside di tipo **inline**



L'elemento in questione si dispone effettivamente **inline** rispetto al suo contenitore, ma non si distribuisce nel flusso che il contenitore genera (come farebbe invece un elemento con **display-inside** di tipo **flow**), bensì si comporta, in un certo senso, come se fosse un unico grosso simbolo o carattere nello scorrimento del testo.

L'effetto è simile a quello che accade se inseriamo un'immagine all'interno di un paragrafo nel mezzo del testo, a meno di non renderla «fluttuante» tramite la proprietà **float**.

CSS e il display di tipo flow

Il display-inside di tipo **flow** è quello di default di tutti gli elementi HTML (con alcune eccezioni per elementi che prevedono una struttura particolare, come le tabelle). Ciascun elemento ha poi un display-outside predefinito, di tipo **block** o **inline**, che regola come l'elemento si comporta all'interno del flusso del testo.

Proprio perché **flow** è il display di default è raramente necessario specificarlo esplicitamente; nel caso però ci serva farlo (ad esempio, per «sovrascrivere» una specifica CSS preesistente), scriveremo qualcosa come:

display: inline flow;

oppure

display: block flow;

Dentro a un contenitore con display-inside di tipo **flow**, la disposizione è regolata dal display-outside dei suoi elementi interni.

Ad un elemento interno con display-outside pari a **block** viene assegnato un riquadro che occupa tutto il contenitore in orizzontale; in altezza gli viene dato lo spazio che gli è necessario. Si pensi per analogia a un paragrafo in un testo.

Un elemento interno con display-outside pari a **inline** è invece considerato essere una sequenza di testo e/o simboli che scorre (almeno per le lingue indoeuropee) da sinistra a destra, su più righe orizzontali, che occupano la larghezza del contenitore; nel caso di due elementi inline consecutivi il secondo inizierà nel punto in cui finisce il primo, proseguendo la riga fino al fondo ed eventualmente riprendendo sulla riga successiva.

È possibile portare uno degli elementi in un contenitore **flow** «fuori» dal flusso delle righe, costringendole a girarci intorno. Per ottenere questo comportamento sarà necessario impostare la proprietà **float**:

float: left;

oppure

float: right;

L'elemento fluttuerà sulla sinistra oppure sulla destra, e il resto del contenuto gli scorrerà intorno.

CSS display-inside: flex

Il display-inside di tipo **flex** dispone i propri elementi come sequenza di riquadri in riga o in colonna, adattandone le dimensioni a seconda della necessità.



CSS e il display di tipo flex

Per una guida completa a tutte le proprietà di CSS che permettono di configurare il display di tipo **flex** di un elemento, si può consultare:

<https://css-tricks.com/snippets/css/a-guide-to-flexbox/>

Il display-inside di tipo **flex** si imposta con:

display: inline flex;

oppure

display: block flex;

È anche possibile scrivere soltanto:

display: flex;

che assegna implicitamente il valore **block** a display-outside.

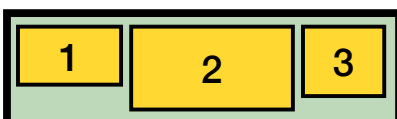
Il display di tipo **flex** ignora il display-outside dei propri contenuti: a ciascuno di essi viene assegnato un riquadro.

Le dimensioni del riquadro dipendono da ciò che quell'elemento a sua volta contiene, e dalle proprietà CSS che regolano il loro allineamento reciproco, nonché la disponibilità ad ampliarsi o rimpicciolirsi, proprietà illustrate nelle prossime slide).

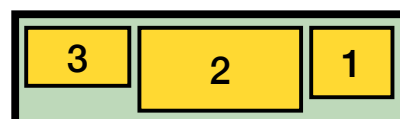
I riquadri vengono poi disposti in riga o in colonna in base alla proprietà **flex-direction**.

CSS display-inside: flex

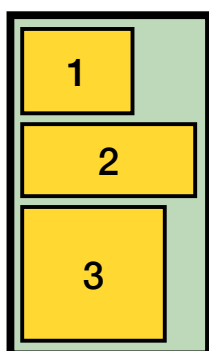
Una volta impostato il display-inside a **flex** è possibile utilizzare una serie di proprietà CSS specifiche di questo display; la disposizione finale nel contenitore dipende dai valori di queste proprietà, sia relative al **contenitore stesso**, sia agli elementi in esso contenuti.



`flex-direction: row;`

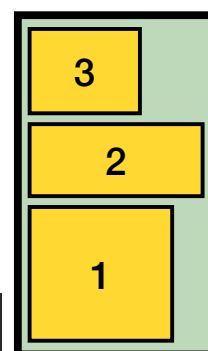


`flex-direction: row-reverse;`



`flex-direction: column;`

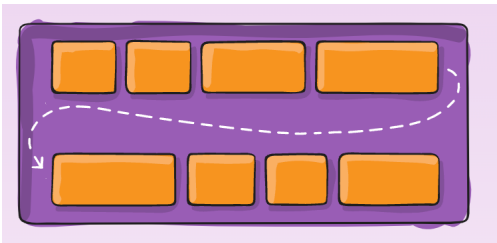
`flex-direction: column-reverse;`



CSS

display-inside: flex

Una volta impostato il display-inside a **flex** è possibile utilizzare una serie di proprietà CSS specifiche di questo display; la disposizione finale nel contenitore dipende dai valori di queste proprietà, sia relative al **contenitore stesso**, sia agli elementi in esso contenuti.

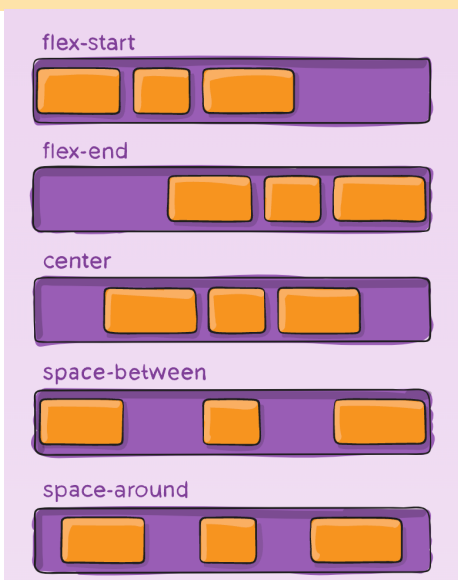


```
flex-wrap: true;  
  
/* con nowrap sborda a destra */
```

CSS

display-inside: flex

Una volta impostato il display-inside a **flex** è possibile utilizzare una serie di proprietà CSS specifiche di questo display; la disposizione finale nel contenitore dipende dai valori di queste proprietà, sia relative al **contenitore stesso**, sia agli elementi in esso contenuti.

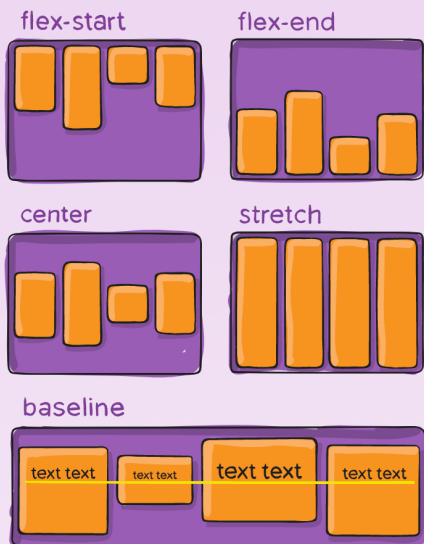


```
justify-content: flex-start;  
/* allinea a sinistra, o in alto se */  
/* il flex-box è in colonna */  
  
/* flex-end allinea a destra o in basso */  
  
/* center allinea al centro */  
  
/* space-between distribuisce lo spazio */  
/* in eccesso tra un elemento e l'altro */  
  
/* space-around distribuisce lo spazio */  
/* in eccesso equamente fra gli elementi */
```

CSS

display-inside: flex

Una volta impostato il display-inside a **flex** è possibile utilizzare una serie di proprietà CSS specifiche di questo display; la disposizione finale nel contenitore dipende dai valori di queste proprietà, sia relative al **contenitore stesso**, sia agli elementi in esso contenuti.



```
align-items: flex-start;
/* allinea a in alto, o a sinistra se */
/* il flex-box è in colonna */

/* flex-end allinea in basso, o a destra */
/* se il flex box è in colonna */

/* center allinea al centro */

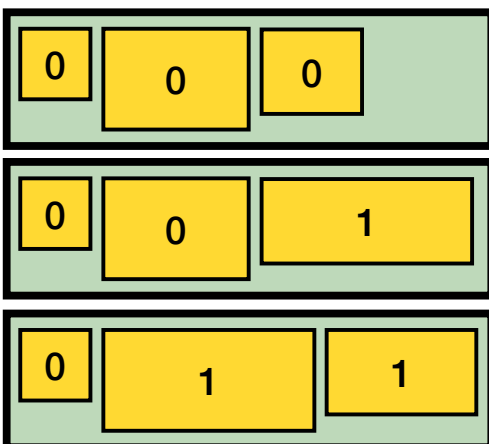
/* stretch allunga gli elementi a */
/* riempire l'altezza (o li allarga a */
/* riempire la larghezza)

/* baseline allinea gli elementi alla */
/* linea di base del testo */
```

CSS

display-inside: flex

Una volta impostato il display-inside a **flex** è possibile utilizzare una serie di proprietà CSS specifiche di questo display; la disposizione finale nel contenitore dipende dai valori di queste proprietà, sia relative al contenitore stesso, sia agli **elementi in esso contenuti**.



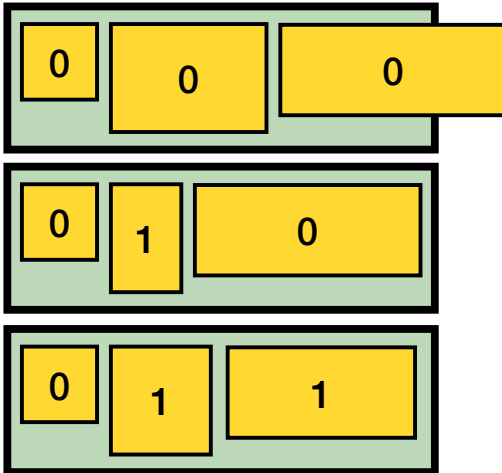
```
flex-grow: 1;
/* ammette valori >= 0. */
/* dice se, e in che proporzione, questo */
/* elemento è disposto ad allargarsi per */
/* occupare lo spazio disponibile */

/* 0 è il default */
```

CSS

display-inside: flex

Una volta impostato il display-inside a **flex** è possibile utilizzare una serie di proprietà CSS specifiche di questo display; la disposizione finale nel contenitore dipende dai valori di queste proprietà, sia relative al contenitore stesso, sia agli **elementi in esso contenuti**.



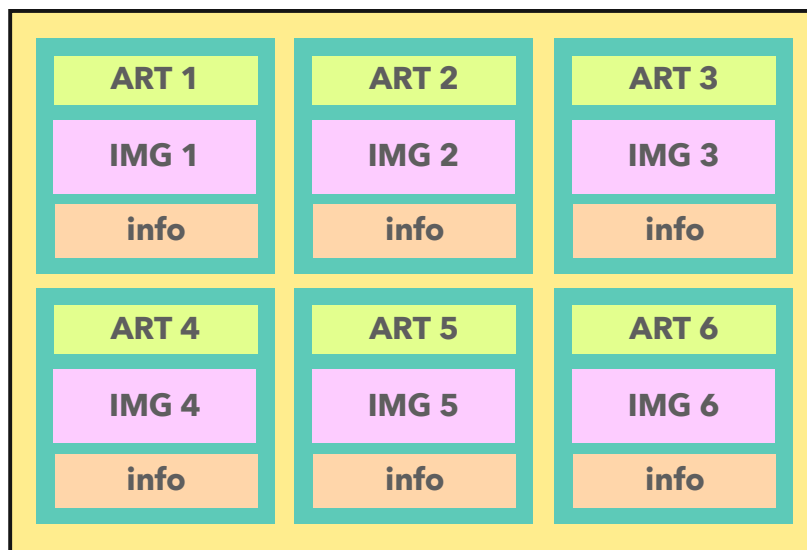
```
flex-shrink: 1;
/* ammette valori >= 0. */
/* dice se, e in che proporzione, questo */
/* elemento è disposto a restringersi per */
/* non eccedere lo spazio disponibile */

/* 0 è il default */
```

CSS

display-inside: grid

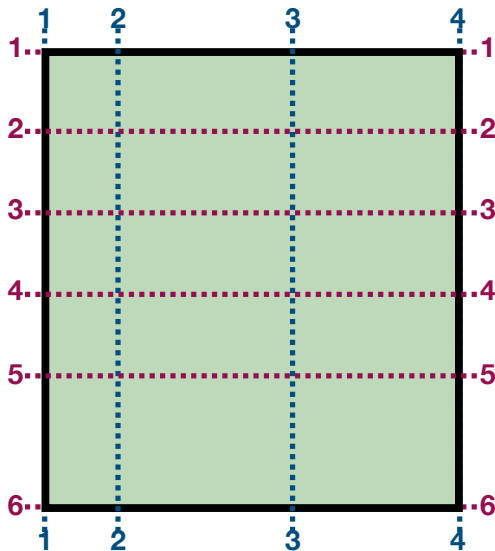
Il display-inside di tipo **grid** definisce una griglia (ottenuta incrociando righe e colonne); ciascun elemento prenderà il posto di una o più celle contigue della griglia.



CSS

display-inside: grid

La griglia viene configurata tramite le due proprietà **grid-template-rows** e **grid-template-columns**, che **elencano** rispettivamente le **altezze** delle righe e le **larghezze** delle colonne

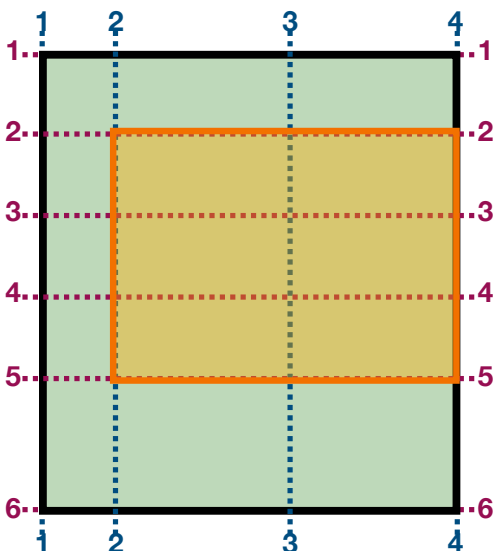


```
grid-template-rows: 50px 1fr 2fr;  
grid-template-columns: repeat(4, 1fr) 10vw;  
/* l'unità di misura fr (=frazione) */  
/* permette di esprimere le dimensioni */  
/* in maniera proporzionale. */
```

CSS

display-inside: grid

Sta a ciascun elemento interno dichiarare quali celle della griglia occuperà, usando le proprietà CSS **grid-row-start**, **grid-row-end**, **grid-column-start**, **grid-column-end**



```
grid-row-start: 2;  
grid-row-end: 5;  
grid-column-start: 2;  
grid-column-end: 4;  
  
/* o anche... */  
  
grid-row-start: 2;  
grid-row-end: span 3;  
grid-column-start: 2;  
grid-column-end: span 2;
```

CSS e il display di tipo grid

Per una guida completa a tutte le proprietà di CSS che permettono di configurare il display di tipo **grid** di un elemento, si può consultare:

<https://css-tricks.com/snippets/css/complete-guide-grid/>

Il display-inside di tipo **grid** si imposta con:

display: inline grid;

oppure

display: block grid;

È anche possibile scrivere soltanto:

display: grid;

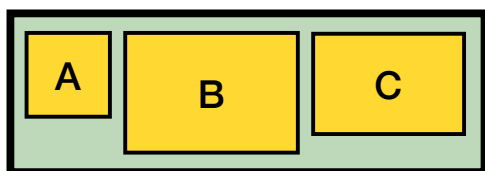
che assegna implicitamente il valore **block** a display-outside.

Il display di tipo **grid** ignora il display-outside dei propri contenuti: a ciascuno di essi viene assegnata l'area della griglia da essi richiesta.

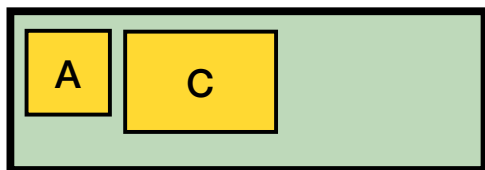
Ulteriori proprietà sia dell'elemento contenitore, sia degli elementi in esso contenuti, permettono di configurare aspetti come l'allineamento reciproco degli elementi in una medesima riga/colonna della griglia, l'allineamento di ciascun elemento nell'area della griglia che gli compete, ecc.

CSS display: none

La proprietà **display** può anche essere impostata al valore singolo **none**. L'effetto in questo caso è di rimuovere l'elemento dalla visualizzazione.



```
B {  
  display: none;  
}
```



Strumenti per CSS

SCSS/SASS

SCSS (Sassy CSS) è un linguaggio per descrivere regole di stile in modo un po' più evoluto di quanto si faccia in CSS

SASS è un pre-processore usato per **tradurre** i fogli di stile dal formato **.scss** al formato **.css** (giacché il browser interpreta solo quest'ultimo)

Alcuni vantaggi di **SCSS**:

- Si possono definire **costanti** da riutilizzare in diversi punti del foglio di stile
- Un foglio di stile può essere incluso in molteplici altri per poter essere riutilizzato
- Una regola può essere scritta **dentro** un'altra: questo genera automaticamente una composizione fra i rispettivi selettori.



Strumenti per CSS

Bootstrap

Bootstrap (v5.3) è un insieme di **css** e **javascript** che definisce degli **stili** e dei **comportamenti** di default per molti comuni componenti di una UI

È possibile usare solo la parte **css** di Bootstrap (quindi, appoggiarsi ai suoi stili predefiniti per definire la propria UI da zero) così come utilizzare i suoi componenti predefiniti (bottoni, menu a tendina, finestre modali...).

Uno stile predefinito non è altro che una **classe CSS**. Usare Bootstrap significa conoscere le classi predefinite che esso fornisce, ed utilizzarle nel proprio HTML

Alcuni vantaggi di **Bootstrap**:

- Evita di scrivere codice per gli elementi più frequentemente utilizzati delle UI
- Semplifica la realizzazione di UI **responsive**, ossia che si adattano alla dimensione dello schermo (tablet, smartphone, computer...)

Svantaggi:

- Le UI fatte con bootstrap sono un po' tutte fatte con lo stampino
- La customizzazione (al di là di cambiare qualche colore e qualche font) non è semplicissima



Strumenti per CSS

Tailwind

Tailwind rovescia il modo di costruire, e collegare fra loro, HTML e CSS.

Tradizionalmente: HTML esprime i contenuti, caratterizzandoli semanticamente attraverso le classi. I fogli di stile dipendono fortemente dalla struttura logica dell'HTML e dalle classi in esso definite.

Tailwind: I CSS definiscono classi generiche, dette **di utilità**: ciascuna classe esprime un'impostazione grafica ben definita, come «bg-white» o «font-semibold». È l'HTML a dipendere dal CSS, dovendosi basare sulle classi in esso definite. Il CSS viceversa è estremamente riutilizzabile.

Svantaggi:

- Uno, **significativo**: Il CSS è più riutilizzabile fra HTML diversi, ma l'HTML è più verboso e ripetitivo. Questo svantaggio è molto meno evidente se ci appoggiamo a qualche framework basato su componenti (ne parleremo più avanti)

